

Introduction

Some LeetCode problems test whether you know a data structure. Others test something more fundamental: whether you can look past the obvious brute-force instinct and find the one property of the problem that makes the whole thing collapse into something trivial. LeetCode 3876, Construct Uniform Parity Array II, is firmly in the second category.

On the surface it looks like a combinatorial search problem — for every number you get two choices, keep it or subtract something from it, and you need every resulting value to share the same parity. The naive brain immediately reaches for trying combinations. But the array can hold up to 100,000 elements, and two choices per element means 2^n possibilities, a number that blows past anything a computer could check in the lifetime of the universe for n much smaller than 100,000.

This is exactly the kind of problem interviewers like, because it rewards a specific skill: recognizing that a search space you can't brute force is often hiding a much smaller, provable structure underneath. Once you see that structure here, the code is four lines. Getting there is the entire exercise, and that's what makes it worth learning properly rather than memorizing.

Problem Overview

You're given an array of n distinct integers, `nums1`. Your job is to build a second array of the same length where every position is filled according to one of two rules:

- Keep the original value as-is, or
- Replace it with the difference between it and some *other, smaller* value in the array (the subtraction has to produce a result of at least 1)

The constraint is that once you're done, every single value in the new array must share the same parity — either they're all odd, or they're all even. You don't have to construct the array itself. You only need to answer yes or no: is it possible?

Because all values in `nums1` are distinct, there's a useful guarantee baked in — if there's an odd number in the array, there's a well-defined smallest odd number with no ties, which turns out to matter a lot once you get to the solution.

Example

Take `nums1 = [1, 4, 7]` .

- Keep `1` — it's already odd.
- Take `4` and subtract `1` : `4 - 1 = 3` , which is odd.
- Keep `7` — already odd.

Every value in the resulting array `[1, 3, 7]` is odd, so the answer is `true` .

Notice what made this work: the only value that needed transforming was `4` , and it only needed to borrow from *one* other number, `1` , to flip into the right parity. That single interaction is the whole problem in miniature.

Intuition

The instinct most people reach for first is to treat this as a search: for each element, try "keep" and try "subtract from every possible partner," then check if any full combination produces uniform parity. That's the brute-force mental model, and it's the one to explicitly reject before writing any code, because with up to 100,000 elements it's 2^{100000} combinations — not slow, but literally uncomputable.

So instead of asking "what combination works," ask a smaller question: what does subtraction actually *do* to parity?

There are only two cases:

- Subtracting two numbers of the **same parity** always gives an **even** result (odd – odd = even, even – even = even).
- Subtracting two numbers of **different parity** always gives an **odd** result (odd – even = odd, even – odd = odd).

That's it — that's the entire mechanism. It doesn't matter *which* odd number or *which* even number you pick, only what parity it has.

From here, a few things fall out immediately:

- An odd number can only turn *even* by subtracting a smaller *odd* number from it.
- An even number can only turn *odd* by subtracting a smaller *odd* number from it.

Now ask: if the array has any odd number at all, can we ever make everything even? No — the *smallest* odd number in the array has no smaller odd number to borrow from, so it's permanently stuck as odd. That kills the "all-even" target the moment a single odd number exists.

So the only target worth checking is "make everything odd." And since every even number can only flip to odd by subtracting a smaller odd number, and there's exactly one smallest odd number available to serve as a partner, the entire multi-way search collapses into a single comparison: **is every even number bigger than the smallest odd number in the array?** If yes, every even number has something smaller to subtract and flip parity. If any even number is smaller than the smallest odd number, it has nothing valid to subtract from, and the answer is `false`.

Brute Force Solution

Idea: For every position, try both choices — keep the value, or subtract every other smaller value from it — and check if any full assignment produces uniform parity.

Algorithm: 1. For each element, generate the set of possible resulting values (itself, plus itself minus every smaller element). 2. Recursively try every combination of choices across all positions. 3. For each complete combination, check whether every resulting value shares one parity. 4. Return `true` if any combination works.

Advantages: Conceptually simple, directly mirrors the problem statement, easy to reason about correctness for tiny inputs.

Disadvantages: Completely infeasible at scale — it doesn't just run slowly, it can't run at all for realistic `n`.

Complexity: Time is $O(2^n \cdot n)$ in the best case (n choices per element, n elements to verify), space is $O(n)$ for recursion depth. For `n = 100,000` this is not a number you can wait out.

Optimal Solution

The optimal approach uses the parity rule from the intuition section directly, and needs exactly one pass to gather information plus one pass to verify.

Step 1 — Split off the odd numbers. Pull every odd number in `nums1` into its own list using a list comprehension.

Step 2 — Handle the "no odd numbers" case. If that list is empty, the array is already all even — return `true` immediately. There's nothing to check.

Step 3 — Find the smallest odd number. Since all values are distinct, `min()` on the odds list gives a single, unambiguous smallest odd value. This is the only number in the entire array capable of flipping any even number's parity.

Step 4 — Verify every even number can reach it. Walk through the original array. Any even number smaller than the smallest odd number can never become odd — there's no valid subtraction available to it — so return `false` the moment you find one.

Step 5 — If nothing failed, return true. If every even number in the array is at least as large as the smallest odd number, every one of them has a valid subtraction partner, so the whole array can become uniformly odd.

Value type	Can it become odd?
Already odd	Yes — no change needed
Even, and larger than smallest odd	Yes — subtract the smallest odd number
Even, and smaller than smallest odd	No — nothing smaller and odd exists to subtract

This table is the entire algorithm. Everything else is bookkeeping.

Python Code

```
def canConstruct(nums1: list[int]) -> bool:
    odds = [x for x in nums1 if x % 2 == 1]

    # No odd numbers at all — array is already uniformly even.
    if not odds:
        return True

    smallest_odd = min(odds)

    # Any even number smaller than the smallest odd has nothing to subtract.
    for x in nums1:
        if x % 2 == 0 and x < smallest_odd:
            return False

    return True
```

Code Walkthrough

- `odds = [x for x in nums1 if x % 2 == 1]` — one pass over the array, collecting every odd value into a new list. This is the first and only place we touch parity classification directly.
- `if not odds: return True` — the early exit for the case where the array is already fully even. No further work is needed because there's nothing that could ever conflict.
- `smallest_odd = min(odds)` — a single call that identifies the only number in the array that matters for the rest of the check. Because values are distinct, this is unambiguous.

- The `for x in nums1` loop is the verification pass. It checks every even number against `smallest_odd` and bails out the instant it finds one that can't be fixed.
- `return True` at the end means the loop completed without finding a disqualifying even number, so the array can be made uniformly odd.

Dry Run

Take `nums1 = [1, 4, 7]` again, this time tracing the code line by line.

1. `odds = [x for x in nums1 if x % 2 == 1] → odds = [1, 7]`.
2. `odds` is not empty, so we skip the early return.
3. `smallest_odd = min([1, 7]) → smallest_odd = 1`.
4. Loop through `nums1`:
5. `x = 1`: odd, skip.
6. `x = 4`: even, is `4 < 1`? No. Continue.
7. `x = 7`: odd, skip.
8. Loop finishes without returning `False`.
9. Return `True`.

This matches the expected answer, and it matches the manual reasoning from the example: `4` only needed to beat `1` in size to have a valid subtraction partner, and it does.

Now a failing case: `nums1 = [3, 6, 1, 2]`.

1. `odds = [3, 1]`.
2. `smallest_odd = min([3, 1]) = 1`.
3. Loop: `x = 3` odd, skip. `x = 6` even, `6 < 1`? No. `x = 1` odd, skip. `x = 2` even, `2 < 1`? No.
4. Return `True`.

And a case that fails: `nums1 = [0, 5]` — wait, values must be positive per typical constraints, so use `nums1 = [4, 9, 6, 2]`. Wait — check: `odds = [9]`, `smallest_odd = 9`. Loop: `4` even, `4 < 9 → True → return False`. Correct, since `4`, `6`, and `2` are all smaller than the only odd number available and have nothing valid to subtract.

Complexity Analysis

Time: $O(n)$. Building the `odds` list is one full pass, `min()` over it is at most another $O(n)$, and the verification loop is a third $O(n)$ pass. Three linear passes still sum to $O(n)$ — there's no nesting between them.

Space: $O(n)$ in the worst case, for the `odds` list, since every value in the array could be odd.

These bounds are tight because the algorithm fundamentally needs to look at every element at least once to classify its parity — there's no way to answer the question without that, so $O(n)$ is also the theoretical floor for this problem.

Alternative Solutions

You could merge the "collect odds" and "find minimum" steps into a single pass using `float('inf')` as a running minimum tracker, avoiding the intermediate `odds` list entirely:

```
def canConstruct(nums1: list[int]) -> bool:
    smallest_odd = min((x for x in nums1 if x % 2 == 1), default=None)
    if smallest_odd is None:
        return True
    return all(x >= smallest_odd for x in nums1 if x % 2 == 0)
```

This keeps the same $O(n)$ time but drops the $O(n)$ space for the odds list down to $O(1)$ extra space, since `min()` with a generator expression doesn't materialize a list. It's a reasonable production-quality alternative if memory matters more than a small readability tradeoff — the two-list version reads more clearly for anyone new to the parity trick.

Edge Cases

- **Empty array:** Trivially satisfies "uniform parity" since there's nothing to conflict — behaves correctly by falling through to `True`.
- **Single element:** As covered below, no special casing is needed; a lone value has nothing to compare against and always passes.
- **All values already odd:** `smallest_odd` gets set, the verification loop finds no even numbers to check, returns `True`.
- **All values already even:** `odds` is empty, returns `True` on the early exit.
- **Smallest odd number is also the smallest number overall:** Every even number is automatically larger, so it always passes.
- **Largest possible array (100,000 elements):** No behavioral edge case, but worth confirming the solution stays linear and doesn't silently degrade — it does not, given the analysis.

above.

Common Mistakes

1. **Trying to brute-force every keep/subtract combination.** This looks correct on paper but is computationally impossible for anything beyond a handful of elements.
2. **Forgetting that "all-even" is never reachable once an odd number exists.** Some solutions waste time checking both targets when only one is ever viable.
3. **Comparing every even number against every odd number instead of just the smallest one.** This turns an $O(n)$ solution into an $O(n^2)$ one for no benefit — only the smallest odd number can ever serve as a valid subtraction partner.
4. **Adding an explicit `if n == 1: return True` guard.** It's unnecessary; a single element trivially satisfies uniform parity through the same logic that handles everything else, and adding the guard signals the general case isn't fully trusted.
5. **Using `<=` instead of `<` when comparing an even number to the smallest odd number.** Since all values are distinct, an even number can never equal the smallest odd number, but getting the comparison direction wrong (or assuming equality could happen) is a common off-by-one style slip.
6. **Forgetting the subtraction result must be at least 1.** This is implicitly satisfied because subtracting a *smaller* value from a larger one always yields a positive difference, but it's worth explicitly confirming rather than assuming.

Interview Questions

- Why does the "all-even" target never need to be checked once any odd number is present?
- Why does only the *smallest* odd number matter, rather than checking against all odd numbers?
- How would the solution change if values in the array were allowed to repeat?
- Can you prove why subtracting an odd number always flips parity, regardless of which odd number is used?
- What's the time and space complexity, and can the space be reduced further?
- How would you modify this to also *construct* the actual output array, not just answer true/false?

Similar Problems

- **LeetCode 3875, Construct Uniform Parity Array I** — the predecessor problem, typically with smaller constraints that allow a more direct simulation before the $O(n)$ insight becomes necessary.

- **Two Sum** — another problem where the "obvious" nested-loop approach is replaced by a single pass once you find the right structural insight (a hash set instead of a parity minimum).
- **Minimum Absolute Difference** — relies on the same idea that sorting or finding extremal values often replaces exhaustive pairwise comparison.
- **Find the Duplicate Number** — another case where a single well-chosen value (rather than a full search) resolves the entire problem.

These are worth practicing together because they all train the same muscle: replacing pairwise comparison with a single extremal value once you understand the underlying rule.

Key Takeaways

- Parity, not the actual magnitude of the numbers, is the only thing that matters here.
- Subtraction between same-parity numbers always produces even results; between different-parity numbers, always odd.
- Once an odd number exists, "all-even" is permanently unreachable, so only "all-odd" needs checking.
- The entire multi-way search collapses into finding one minimum value and comparing every even number against it.
- The result is $O(n)$ time and $O(n)$ space (or $O(1)$ extra space with a single-pass variant), replacing an exponential brute force.

Watch the Video

If you'd rather see this reasoned through out loud — including the real timing comparison between brute force and the optimal solution, and two quiz rounds to test your understanding — the full walkthrough is here: {LINK}

About the Series

This breakdown is part of the *Fun with Learning Technology* series, where each episode takes one real coding interview problem and works backward from a naive first instinct to a provably correct, efficient solution — the same way you'd actually reason through it under interview pressure, not a polished answer pulled from memory.

Call To Action

If this helped the problem click, the biggest favor you can do is watch the full video on YouTube and hit subscribe — it's what keeps this series going. Subscribe to the Substack for the write-up version of every problem before it's even published, drop a comment with your own approach or a question you got stuck on, and share this with anyone prepping for interviews right now. Every bit of that helps more than you'd think.

Quick Review

When array must alternate/match parity pattern, what's the trigger?

Parity (odd/even) problems reduce to checking mod-2 values, not actual numbers — think bitmask or parity array.

Time complexity for scanning array once to fix parity mismatches?

$O(n)$ — single pass comparing each element's parity against the target pattern.

Space complexity for building the uniform parity output array?

$O(n)$ for the output array itself; $O(1)$ extra if modifying in place.

Common bug when checking 'same parity' via subtraction?

Forgetting subtraction preserves parity only for the difference, not the original values — $(a-b)$ even means a, b share parity.

Same Parity vs Different Rules — what's the key distinction?

Same Parity checks $a\%2==b\%2$ directly; Different Rules problems apply a transform (like division) before comparing parity.

Interview follow-up: how would you verify parity in $O(1)$ per element?

Use bitwise AND with 1 ($\text{num} \& 1$) instead of modulo — faster and avoids sign issues with negative numbers.